Rapport WeJob

Par Arthur MILLET, Akram ROUBIA, Chloé MERCIER, Jessy SANFILIPPO

1. Contexte et enjeux

Le projet WeJob existait déjà avant notre intervention, mais il se heurtait à de gros blocages techniques et fonctionnels.

À notre arrivée sur le projet, le constat était assez clair : la plateforme souffrait de “maux chroniques” qui freinaient son adoption. Nous avons identifié trois problèmes majeurs : une latence trop élevée, un matching souvent peu pertinent entre les offres et les candidats (entraînant de faibles conversions), et une expérience utilisateur (UX) globale assez frustrante.

Pour les entreprises partenaires, ces défauts se traduisaient concrètement par des opportunités manquées et une perte de compétitivité sur le marché. Notre travail n’était donc pas de repartir de zéro, mais de réduire activement ces problèmes. L’objectif principal de notre mission a été d’utiliser l’intelligence artificielle pour améliorer la pertinence du matching et fluidifier l’ensemble du parcours.

Glossaire

- **vLLM** : Serveur haute performance pour l’inférence de modèles de langage.
- **Similitude Cosinus** : Mesure mathématique utilisée pour comparer l’orientation de deux vecteurs
- **Small Language Models (SLM)** : Modèles d’IA optimisés pour des tâches spécifiques, moins gourmands en ressources que les modèles généralistes.
- **LPD / nLPD** : Loi fédérale sur la Protection des Données (Suisse).
- **Fallback** : Mécanisme de secours qui prend le relais en cas d’échec d’un système automatisé.
- **Matching** : Fait d’associer deux entités ensemble. Ici par exemple, un candidat avec une offre d’emploi.
- **Hardskills / Softskills** : Compétence technique / Qualités personnelles, compétences sociales.
- **React** : Framework Javascript.
- **Laravel** : Framework PHP.
- **Docling** : Bibliothèque Python permettant l’utilisation rapide de modèles d’IA pour l’extraction de données.
- **Hallucination** : Information inventée, fausse, affirmée par un LLM.
- **Workflow** : Suite de tâche effectuées, flux de travail.
- **Log** : Fichier contenant des données pouvant servir à contextualiser un événement qui s’est produit à un moment donné
- **JSON** : Format de données structuré
- **Agent (IA)** : Unité logicielle spécialisée qui utilise l’intelligence artificielle pour accomplir de manière autonome une tâche précise au sein d’un processus plus large.
- **PV (Planned Value)** : Valeur Planifiée, ce qu’on avait prévu de faire.
- **EV (Earned Value)** : Valeur Acquise, ce qu’on a réellement fait (en argent).
- **SPI (Schedule Performance Index)** : Indice de Performance des Délais, ratio qui indique si on est en avance ou en retard sur le planning.
- **MVP** : produit minimal viable

1.1 L’équipe derrière le projet

Pour relever les défis de WeJob, notre équipe s’est structurée autour de quatre profils complémentaires :

- **Arthur MILLET** *(Chef de projet)* : Fullstack, IA
- **Akram ROUBIA** : Fullstack, IA
- **Chloé MERCIER** : Fullstack
- **Jessy SANFILIPPO** : Backend, IA

L’équipe inclut également les clients : **Ruben OLENDER**, ainsi que **Mélany BARRÓN**.

1.2 Personas et besoins utilisateurs

Pour guider notre développement, nous avons défini plusieurs personas représentant la diversité du marché suisse :

Jules (23 ans, étudiant) : Cherche un stage d'ingénieur de 22 semaines, bien rémunéré et axé sur les technologies web (React, TypeScript). Ses frustrations incluent le mélange des types de contrats (CDI/Stages) et le manque de précision des offres.

Marc (54 ans, reconversion) : Ancien conseiller bancaire souhaitant s'orienter vers l'informatique. Il peine à trouver des offres sans prérequis techniques massifs et se sent discriminé par son âge.

Louis (35 ans, recruteur) : Besoin de publier rapidement des offres et de filtrer efficacement les CV par compétences techniques réelles.

Olivier (38 ans, formateur) : Veut améliorer la visibilité de son établissement mais trouve que les plateformes actuelles ne différencient pas assez les offres d'emploi des offres de formation.

2. Gestion, planification et suivi

2.1 Gestion de projet - Collaboration

Le projet a été piloté via une méthodologie Agile rigoureuse :

- Réunions quotidiennes *(Daily)* et hebdomadaires *(Weekly)* pour synchroniser les équipes.
- Sprints thématiques gérés sur Jira.
- Pair programming pour les parties les plus critiques du code.
- Code Review pour verifier que le code est correct, maintenable, lisible, performant et conforme aux standards de l'équipe.
- Utilisation intensive de Git : Gestion par branches spécifiques pour chaque fonctionnalité (*feat/*, *fix/*, *dev_matching_*) afin d'éviter les régressions et faciliter la collaboration distante.

2.2 Chronologie des Sprints

Le projet s'est étalé sur environ deux mois début 2026. En première approche, les sprints ont été définis ainsi :

2.2.1 Planification théorique

Sprint 0 (12/01 - 25/01) : Conception, prise en main de l'architecture et réflexion sur le système IA.

Sprint 1 (26/01 - 01/02) : Modification de la base de données, Modification du back et du front pour une utilisation optimale de la base de données et rédaction des premiers prompts IA.

Sprint 2 (02/02 - 08/02) : Mise en place de l'architecture multi-agents, ajouts de composants IA, premier calcul du score de matching et visualisation des résultats par l'utilisateur.

Sprint 3 (09/02 - 15/02) : Implémentation du système multi-agent pour le plan de carrière et ajout des fonctionnalités permettant aux candidats d'avoir leur plan de carrière.

Sprint 4 (23/02 - 08/03) : Amélioration du matching et plan de carrière, correction des derniers bugs, rédaction de la documentation et implémentation d'un support d'aide basé sur l'IA.

2.2.2 Planification finale

Le projet a subi de léger retards, dûs à une mauvaise architecture de la base de données. Un réajustement du scope du projet, ainsi que des sprints ont été nécessaires.

Les sprints 0 et 1 ont été respectés dans les temps.

Sprint 2 - Matching (02/02 - 08/02) Calcul du score de matching, visualisation du score de matching pour chaque offre d'emploi et redéfinition du profil candidat.

Sprint 3 - Réparations (09/02 - 15/02) Extraction des CVs, mise en place de l'architecture multi-agents et réparation du formulaire profil candidat.

Sprint 4 - Finalisation (23/02 - 08/03) Réparation du formulaire d'emploi, finalisation du workflow pour le matching et rédaction de la documentation.

2.3 Indicateurs de performance

Nous avons suivi la santé du projet via des indicateurs classiques :

Nous avons chiffré le projet en prenant comme base les éléments suivants : - 4 étudiants - 220h de travail sur le projet - taux journalier moyen 400 CHF par personne (pour une journée de 8h)

Coût = 4 * 220 * (400/8) = 44000 CHF ≈ 48000 €

- **PV** : 48 000 €
- **EV** : 48 000 * 0.77 ≈ 36800 €
- **SPI** : = EV/PV = 36800/48000 ≈ **0.77**

Un SPI inférieur à 1 indique un léger retard accumulé. Celui-ci a commencé à partir du 3ème sprint, principalement dû à des incohérences relevées en base de données pour la définition d'un profil candidat. Régler ces incohérences nous a obligé à revoir et adapter le schéma de la base de données.

3. Difficultés et risques

3.1 Les défis techniques

L'intégration de l'IA dans un environnement legacy a présenté plusieurs obstacles majeurs.

3.1.1 L'extraction complexe de documents

L'un des défis a été l'extraction des CV. Le format PDF est extrêmement hétérogène, surtout avec l'émergence de designs complexes comme ceux de Canva. Certaines informations étaient parfois mal placées lors de l'extraction, mais l'utilisation de Docling a permis de stabiliser en partie le processus. Quelques étapes supplémentaires telles que le réalignement de certaines parties du texte avec d'autres ont été nécessaires.

3.1.2 La refonte de la base de données

C'est l'un des points les plus critiques de notre mission. La structure initiale de la base de données ne permettait pas de stocker les nuances nécessaires au matching (niveaux requis vs possédés, types d'expérience). Nous avons dû procéder à une normalisation profonde des tables Candidats et Offres, tout en migrant les données existantes sans perte, afin de rendre l'algorithme de matching performant.

3.2 Matrice des risques

Description du risque	Probabilité	Gravité	Prévention	Réparation
Le LLM (ou vLLM) ne fonctionne pas	1/5	4/5	possibilité de désactiver les fonctionnalités IA	Utiliser un autre LLM ou laisser les fonctionnalités IA désactivées
Indisponibilité du serveur IA	1/5	5/5	possibilité de désactiver les fonctionnalités IA	Désactiver les fonctionnalités IA, sauf si on a un second serveur
Conflits de code	3/5	3/5	Mise en place rigoureuse des bonnes pratiques de développement et de travail collaboratif avec Git.	Avec Git, il est possible de revenir en arrière, d'écraser les derniers changements pour revenir à une version stable / fonctionnelle
Changement de périmètre fonctionnel	4/5	3/5	Comprendre les besoins du client et les implicites	Redéfinir les fonctionnalités voulues, les trier par ordre de priorité

4. Enjeux Socio-Environnementaux

4.1 Sobriété Numérique (RGESN)

Nous avons réalisé un audit basé sur le RGESN [**7**] pour limiter notre impact :

Interface et Parcours Utilisateur (UX/UI)

- **Sobriété des contenus** : Nous avons fait le choix de n'utiliser aucun contenu vidéo ou audio. La plateforme repose sur le format le plus léger : du texte accompagné d'icônes ciblées. Les seules animations présentes sont purement fonctionnelles (affichage des réponses).
- **Navigation optimisée** : Pour éviter la surconsommation de données, nous avons banni le défilement infini pour les listes de jobs, de candidats ou de formations, lui préférant un système de pagination. De plus, chaque parcours (recherche, publication, inscription) a été simplifié pour réduire le nombre de clics et de pages chargées.
- **Économie de ressources** : La plateforme ne télécharge qu'une seule police de caractères. Nous utilisons également des composants natifs dès que possible pour limiter le poids des bibliothèques externes.

Efficacité des échanges (Frontend/Backend)

- **Limitation des requêtes inutiles** : C'est un point fort de notre implémentation. Aucune requête n'est envoyée au serveur pendant qu'un utilisateur remplit un formulaire tant que le format attendu n'est pas respecté. La validation se fait côté client, et l'envoi ne se déclenche que sur action explicite de l'utilisateur.
- **Transparence des transferts** : Avant chaque téléchargement de fichier (CV, logo), l'utilisateur est informé des formats compatibles et de la taille maximale. Cela évite les erreurs de transfert répétées qui consomment de la bande passante inutilement.

- **Gestion des services tiers** : Si certains services d'IA sont automatisés (pour le matching), toutes les fonctions de conseil (salaire, préavis, recherche spécifique) ne sont déclenchées que si l'utilisateur le demande explicitement.

Backend et Données

- **Système de Cache** : Nous exploitons le système de cache natif du framework Laravel. Cela permet de servir les données les plus consultées sans solliciter la base de données à chaque fois, réduisant ainsi la charge CPU de nos serveurs.
- **Traitement en arrière-plan** : Pour les tâches lourdes, nous informons systématiquement l'utilisateur qu'un traitement est en cours. Cela évite qu'il ne rafraîchisse la page plusieurs fois par impatience, ce qui générerait des requêtes superflues.
- **Cycle de vie des données** : Nous appliquons des délais de conservation. Avant toute suppression, les données sont anonymisées pour permettre un entraînement futur de nos modèles internes sans porter atteinte à la vie privée.

4.2 Éthique, transparence et conformité

Le projet WeJob ne se contente pas d'être performant, il se veut responsable. Puisque nous touchons au domaine sensible de l'emploi, nous avons placé l'équité et la transparence au centre de notre développement. Cette démarche nous permet d'anticiper notamment les exigences de l'IA Act, le nouveau règlement européen sur l'intelligence artificielle [**8**].

IA Act : Le recrutement comme "Haut Risque" Le nouveau règlement européen (IA Act) classe les systèmes d'IA liés à l'emploi dans la catégorie "Haut Risque". WeJob anticipe ces obligations :

- **Équité par anonymisation** : Nous pratiquons l'anonymisation (nom, âge) avant le matching pour neutraliser les biais discriminatoires.
- **Contrôle Humain (Human-in-the-loop)** : L'IA ne remplace pas l'humain. Elle propose, mais c'est l'utilisateur qui valide, corrige et signe.
- **Transparence** : L'utilisateur sait quand il parle à une IA et comment son score est calculé.
- **Pour l'audit** : chaque entrée, et sortie d'une IA est stockée en base de données, en cas de litige avec un utilisateur.

Cependant, une mise en conformité totale avec l'IA Act pour une mise sur le marché demanderait -entre autre- les étapes suivantes :

- **Documentation technique approfondie** : Nous devrions fournir une documentation détaillée et approfondie de comment le matching assisté par IA fonctionne précisement, comment l'IA élimine les biais, etc. Dans le futur, nous devrions détailler les jeux de données utilisés pour l'entraînement et le fine-tuning de nos modèles (si nous passons à cette étape).
- **Enregistrement dans la base de données de l'UE** : En tant que système "Haut Risque", WeJob devrait, à terme, être enregistré officiellement auprès des autorités européennes avant sa mise sur le marché.

RGPD, LPD et nLPD : Souveraineté des données Opérant sur le marché suisse et européen, WeJob respecte la LPD [**10**], la nLPD [**11**] (Suisse) et le RGPD [**9**] (Europe) :

- **Droit de contrôle** : Chaque candidat a un accès à ses données extraites par l'IA et peut les modifier ou les supprimer.
- **Souveraineté technique** : En utilisant un serveur local (vLLM) pour nos modèles d'IA, nous évitons d'envoyer des données personnelles sensibles vers des API tierces étrangères dont nous ne maîtrisons pas la politique de confidentialité.
- **Minimisation** : Seules les données utiles au matching sont extraites. Le numéro de téléphone ou la localisation ne sont utilisés que pour faciliter l'inscription, sans stockage abusif.

5. Architecture système : Ce qui est déjà présent

5.1 Vision globale (Front & Back)

L'architecture a été pensée pour être modulaire et scalable, séparant nettement les responsabilités :

- Le **Front-end** : Une interface client moderne permettant une navigation fluide et une interface de discussion avec un LLM (Large Language Model).
- Le **Back-end** : Divisé entre un serveur web et un serveur d'application . Le serveur d'application communique avec une base de données MySQL.
- Le **Pôle IA** : Un serveur IA dédié qui gère les traitements lourds (futur matching, future analyse de CV).

5.2 Système Multi-Agents

Au cœur du système réside un Agent Orchestrateur. Son rôle est de spécifier le besoin en fonction de la requête utilisateur pour rediriger vers la bonne fonctionnalité, le bon agent (poster une offre, chercher un job, support, plan de carrière, etc).

6. Approche et technologies utilisées

6.1 Choix de la stack technique

Laravel - PHP (Back-end)

- **Prise en main** : Laravel [**2**] permet d'aller vite avec des conventions simples.
- **Productivité** : Le framework propose énormément de modules prêts à l'emploi.
- **Coût** : L'infrastructure nécessaire pour faire tourner du PHP [**1**] est généralement moins chère et plus facile à déployer.

React - Javascript (Front-end)

- **Fluidité et UX** : React [**3**][**4**] nous permet de créer une interface de type "Single Page Application" (SPA). Cela élimine les rechargements de page intempestifs.
- **Composants réutilisables** : Grâce à sa logique de composants, nous avons pu développer rapidement des éléments d'interface cohérents (cartes d'offres, formulaires de profil, barres de progression) et les réutiliser sur l'ensemble du site, facilitant ainsi la maintenance.

Docling - Python (Traitement IA)

- **Extraction de précision** : Contrairement aux bibliothèques PDF classiques qui perdent souvent la structure du texte, Docling [**5**][**6**] est capable de comprendre les mises en page complexes, ce qui est crucial pour analyser les CV créés sur des outils comme Canva.
- **Conversion structurée** : Il transforme des documents "non structurés" en formats Markdown. C'est cette étape qui permet à nos agents IA de "comprendre" le contenu du CV sans être perturbés par des problèmes de formatage.

6.2 Outils utilisés

Au-delà de la stack technique pure, nous avons utilisé toute une panoplie d'outils pour nous organiser, communiquer et tester nos développements au quotidien.

GitHub (Collaboration et Versioning) C'est le pilier de notre travail d'équipe. GitHub nous a permis de centraliser le code, de gérer les versions via des branches spécifiques et d'éviter les pertes de données. Chaque nouvelle fonctionnalité passait par une "Pull Request", ce qui nous permettait de relire le code des autres avant de l'intégrer à la branche principale.

VS Code (Éditeur de code) Visual Studio Code a été notre outil de travail principal pour l'écriture du code. Sa flexibilité et ses extensions (notamment pour le débuggage PHP et le support de React) nous ont permis d'être beaucoup plus efficaces, peu importe le langage utilisé (PHP, JS ou Python).

Microsoft Teams (Communication) Pour rester synchronisés, surtout lors des phases de travail à distance, Teams a été notre moyen de communication principal. C'est là que nous avons tenu nos réunions quotidiennes (Daily) et hebdomadaires, et que nous avons partagé les documents de spécifications avec les parties prenantes.

XAMPP & phpMyAdmin (Serveur local et BDD) Pour développer sans avoir besoin d'être connectés en permanence à un serveur distant, nous avons utilisé XAMPP. Cela nous a permis de simuler un environnement Apache/MySQL sur nos machines. phpMyAdmin nous a été indispensable pour manipuler la base de données visuellement, tester nos requêtes SQL et vérifier l'intégrité des données après les phases d'extraction IA.

Stripe (Gestion des paiements) La plateforme WeJob intégrant une partie "Pricing", nous avons implémenté Stripe pour gérer les transactions et les abonnements. C'est un choix de sécurité et de simplicité : l'outil gère toute la complexité des paiements réglementés, nous permettant de nous concentrer sur l'expérience utilisateur.

Jira (Pilotage de projet) Enfin, pour la gestion de nos sprints Agile, nous avons utilisé Jira. C'est l'outil qui nous a permis de suivre nos tickets, de mesurer notre avancement et de calculer nos indicateurs de performance (comme le SPI mentionné plus haut).

7. Solutions spécifiques

7.1 Première idée : comparaison vectorielle

7.1.1 Algorithme de Matching Vectoriel

Durant les deux premières semaines du projet, notre priorité a été de définir un algorithme de matching qui soit à la fois objectif, complet et performant. **Nous ne voulions pas d'une simple recherche par mots-clés qui manque souvent de nuance**. Après réflexion, nous avons opté pour une approche par vectorisation.

Transformer les compétences en coordonnées Nous transformons les compétences d'un candidat et les exigences d'une offre d'emploi en vecteurs de scores numériques grâce à un système multi-agent (***Voir 7.1.2***). (par exemple : 0.80, 0.15, 0.50, ...). Chaque dimension du vecteur correspond à une compétence spécifique (Hard Skill, Soft Skill ou Langue), et la valeur associée représente le niveau de maîtrise ou d'exigence attendu.

Le calcul de pertinence Pour déterminer si un candidat est pertinent pour une offre, notre système effectue une double analyse mathématique sur ces vecteurs :

L'alignement (Comparaison Cosinus) : Nous calculons le cosinus de l'angle entre le vecteur Candidat et le vecteur Offre. Plus cette valeur est proche de 1, plus les vecteurs pointent dans la même direction. Cela nous indique si le mix de compétences du candidat correspond à la structure de l'offre.

L'intensité (Comparaison de Norme) : L'alignement ne suffit pas ; il faut aussi que le niveau soit au rendez-vous. Nous comparons donc la longueur (la norme) des vecteurs. Plus la différence de norme est faible, plus le niveau réel du candidat est proche de celui espéré par le recruteur.

Un raffinement pour l'équité Un aspect important de notre logique est le traitement de la sur-qualification. Si un candidat a un score supérieur à celui demandé pour une compétence donnée, nous ramenons artificiellement son score au niveau de l'offre pour le calcul de référence. Cela évite de fausser le score global et garantit que nous cherchons le profil le plus pertinent, et pas seulement celui qui a les chiffres les plus élevés partout.

Le résultat final est un tri par ordre décroissant de score, permettant au recruteur comme au candidat de visualiser immédiatement les meilleures opportunités avec une base mathématique solide.

7.1.2 Système multi-agent : Création des vecteurs candidats

Pour la création de vecteurs à partir d'un CV, nous avons mis en place une chaîne spécialisée :

- **Agent Descriptif** : Analyse le texte pour expliciter les "non-dits" et enrichir le profil du candidat.
- **Agent Créateur JSON** : Transforme cette analyse en un fichier JSON structuré, prêt à être consommé par nos algorithmes de matching.

Le format de sortie est : **{ "Nom de la compétence" : score (entre 0 et 1) }**

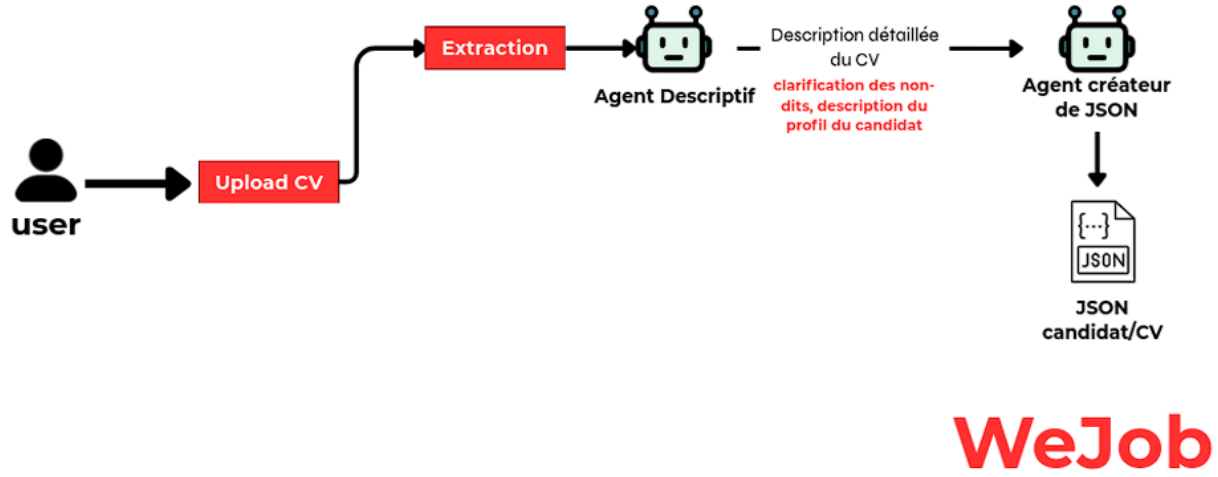

Figure 1: Design simple

Dans notre conception initiale, nous avions envisagé d'ajouter une étape de contrôle qualité via un troisième agent : l'Agent Judge. Son rôle était d'agir comme un superviseur pour garantir l'intégrité des données ; il devait comparer la synthèse de l'Agent Descriptif avec le CV original pour détecter d'éventuelles hallucinations. Si le score de similarité calculé par cet agent était trop faible, il devait renvoyer le texte à l'Agent Descriptif avec des instructions spécifiques pour corriger le tir.

Cependant, après réflexion, nous avons décidé de ne pas implémenter cet agent pour plusieurs raisons stratégiques :

- **La validation par l'utilisateur** : C'est la raison principale. Le système d'auto-remplissage du formulaire d'inscription n'est pas une boîte noire : les informations extraites sont présentées au candidat qui doit impérativement les vérifier, les corriger si besoin, et les valider. Puisque l'utilisateur joue déjà ce rôle de "juge final", l'ajout d'un agent IA pour faire le même travail devenait redondant.
- **Optimisation de la latence** : Chaque appel à un LLM ajoute un temps de traitement non négligeable. En supprimant cet agent, nous accélérons le processus de traitement du CV.

7.1.3 Avantages

- **Précision granulaire** : Contrairement à une simple recherche par mots-clés, cette méthode permet de comparer l'adéquation réelle entre le niveau de maîtrise d'un candidat et les attentes spécifiques de

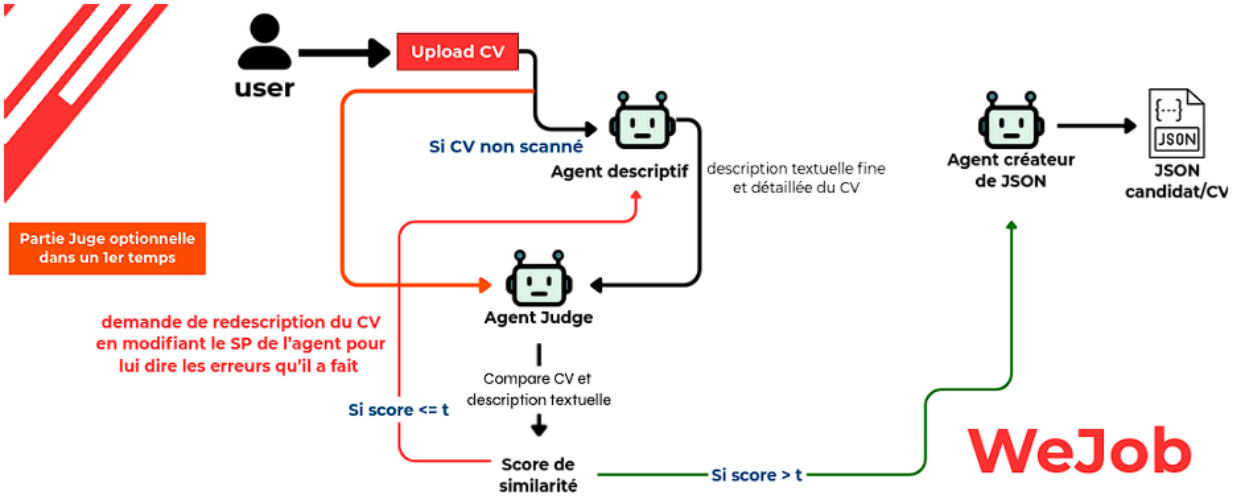

Figure 2: Design juge

l'offre.

- **Performance et scalabilité** : L'algorithme est particulièrement efficace. Si l'on note n la taille des vecteurs (nombre de compétences) et m le nombre d'offres à comparer, la complexité est en O(n×m). Cela garantit des résultats rapides, même avec une base de données volumineuse.

7.1.4 Limites

Cette méthode, bien qu'efficace théoriquement, pose plusieurs problèmes :

- **Difficulté à mettre en place** : La base de données initiale n'était pas structurée pour supporter nativement cette logique vectorielle. Réadapter l'intégralité du Front-end, du Back-end et du schéma de données pour coller parfaitement à cet algorithme aurait largement dépassé le cadre temporel imparti au projet.
- **Pas de pondération** : Dans notre implémentation actuelle, les Hard skills, Soft skills et les langues sont traités au sein d'un vecteur unique. Cela empêche d'attribuer un poids plus important aux compétences techniques (souvent prioritaires) par rapport aux traits de personnalité. Une évolution logique serait de scinder cette analyse en trois vecteurs distincts pour affiner le score final.
- **Pas de prise en compte du niveau d'expérience** : L'algorithme peine à différencier la maîtrise technique brute de la séniorité professionnelle. Un candidat "Junior" peut avoir un excellent score sur une technologie précise (comme React) sans pour autant posséder l'expérience métier globale qu'un recruteur recherche pour un poste senior. C'est une nuance que le vecteur seul, sans métadonnées supplémentaires, ne peut pas encore capter.

7.2 Deuxième idée : Matching simplifié

Les limites techniques rencontrées nous ont conduits à redéfinir l'algorithme de matching. L'idée était de proposer une solution simple mais suffisamment robuste pour être efficace immédiatement. On se rapproche ici d'un matching par mots-clés "intelligent", enrichi par des critères contextuels et une pondération précise.

7.2.1 Un périmètre de données élargi

Pour que le matching ne soit pas qu'une simple comparaison de textes, nous avons intégré de nouveaux paramètres dans le profil et les offres :

- **Mobilité géographique** : Pour vérifier si le candidat est prêt à se déplacer pour le poste (déménagement par exemple).

- **Expérience professionnelle** : Pour filtrer par niveau de séniorité.
- **Niveau d'éducation** : Prise en compte des diplômes requis.
- **Catégories techniques** : Pour un meilleur classement des compétences.
- **Informations de contact** : Relevées pour pré-remplir les formulaires et fluidifier l'inscription.

7.2.2 Système multi-agent : création des listes de mot-clé

Il est important de noter que l'architecture globale du système n'a pas été modifiée : nous avons conservé la même chaîne de traitement, mais nous avons transformé le format de sortie du dernier agent (le Créateur JSON). Auparavant, l'agent générait un dictionnaire associant chaque nom de compétence à un score de maîtrise compris entre 0 et 1.

Désormais, la sortie de l'agent est devenue une véritable "fiche d'identité". Nous extrayons maintenant :

- **Les compétences classées :** Une distinction nette entre les Hard skills et les Soft skills.
- **La maîtrise linguistique :** Les langues parlées sont désormais associées à leur niveau spécifique (A1, C2, etc.).
- **Le parcours et l'expertise :** Le niveau d'études, le nombre d'années d'expérience professionnelle et les champs techniques de prédilection sont identifiés précisément.
- **Les données de contact et logistiques** : Pour fluidifier l'inscription, l'agent récupère directement le numéro de téléphone et la localisation.
- **Le critère de mobilité :** Une information binaire (0 ou 1) qui nous indique immédiatement si le candidat est prêt à se déplacer.

7.2.3 Logique du calcul par catégorie

Nous avons divisé les compétences (Hardskills et Softskills) en deux catégories : Required (obligatoire) et Nice to Have (facultatives).

- **Compétences** : Le score est un ratio. Si une offre exige 5 Hard skills obligatoires et que le candidat en possède 2, il obtient un score de 2/5 pour ce champ. On applique la même logique pour les compétences optionnelles (Nice to Have).
- **Mobilité** : Le système détermine si le candidat se situe dans la même ville, dans le même canton ou dans le même pays que le lieu de travail. Un bonus est accordé si le candidat a explicitement coché "Mobilité géographique" sur son profil.
- **Expérience et Éducation** : Ces critères fonctionnent par "pénalité". Plus l'écart entre le niveau requis par l'offre et celui du candidat est grand (dans le sens où le candidat est en dessous des attentes), plus le score de la catégorie baisse.

7.2.4 Pondération et score global

Le score global du candidat est ensuite calculé selon ces pondérations :

- **Experience** : 0.4
- **Hardskills Required** : 0.3
- **Softskills Required** : 0.1
- **Mobility** : 0.1
- **Hardskills Nice to Have** : 0.075
- **Softskills Nice to Have** : 0.025

La formule de calcul est la suivante :

$$\begin{aligned} \text{score}_{\text{global}} = {} & 0.3 \cdot \text{score}_{\text{HS_Required}} + 0.1 \cdot \text{score}_{\text{SS_Required}} \\ & + 0.075 \cdot \text{score}_{\text{HS_NTH}} + 0.025 \cdot \text{score}_{\text{SS_NTH}} \\ & + 0.4 \cdot \text{score}_{\text{Experience}} + 0.1 \cdot \text{score}_{\text{Mobility}} \end{aligned} \tag{1}$$

où HS : Hard Skills / SS : Soft Skills / NTH : Nice To Have (compétences appréciées).

7.2.5 Avantages

- **Simplicité** : Le système est lisible, tant pour nous que pour l'utilisateur qui peut comprendre son score.
- **Capacité d'évolution** : Les coefficients de pondération peuvent être ajustés en quelques clics pour coller aux futures directions de WeJob.
- **Complet** : En intégrant la mobilité et les années d'expérience, on offre bien plus de contexte que le système précédent.

7.2.6 Limites

- **Précision sémantique** : Comme nous sommes sur un matching par mots-clés, le système peut passer à côté de synonymes si l'IA n'a pas correctement "normalisé" les termes en amont.
- **Manque de nuance sur les niveaux** : On compte les compétences possédées, mais on ne pondère pas encore assez finement le "niveau de maîtrise" au sein de chaque compétence (ex: faire la différence entre un "débutant Python" et un "expert Python" si les deux ont simplement listé le mot-clé).

7.3 Plan de carrière

Bien que nous n'avons pas eu le temps d'implémenter cette fonctionnalité faute de temps, nous avons pensé à deux approches pour le Plan de carrière :

- **Court terme** : Suggérer des formations pour valider une compétence où le candidat est "sous-qualifié" par rapport à une offre visée.
- **Moyen/Long terme** : Discussion avec l'IA pour définir un projet professionnel. L'IA pourrait alors proposer différents postes que le candidat pourrait occuper dans un futur plus ou moins proche. Le but par la suite de l'IA, est de définir les futures compétences nécessaire pour faciliter cette évolution en proposant des formations adaptées.

7.4 Support

De même, bien que nous n'avons pas eu le temps de traiter cette partie, Nous avons pensé à une manière de l'implémenter :

Le Support IA utiliserait une base de questions-réponses (QA). Si une question utilisateur est proche d'une question de la base, l'IA traite la réponse en se basant sur la réponse associée ; sinon, un fallback vers un humain est activé pour répondre. La réponse de l'humain servira à enrichir la base de questions-réponses.

8. Bilan de compétence - Réaliser des solutions numériques

Dans le cadre de ce projet, nous avons mobilisé la compétence « Réaliser des solutions numériques » à travers plusieurs dimensions techniques, méthodologiques et éthiques. Ce bilan synthétise la manière dont nous avons répondu aux exigences du métier d'ingénieur.

1. RÉALISER DES SOLUTIONS NUMÉRIQUES	• En utilisant des méthodes et des outils adaptés • En respectant la réglementation (RGPD, propriété intellectuelle, contrat) et les recommandations (bonne pratique, sécurité, etc.) • En optimisant l'utilisation des ressources matérielles et énergétiques • En collaborant efficacement • En respectant les spécifications et normes techniques • En justifiant ses choix de conception • En s'adaptant à un environnement technologique et stratégique en constante évolution

Figure 3: Compétence choisie

Utilisation de méthodes et d'outils adaptés Pour mener à bien WeJob, nous avons instauré une méthodologie Agile (Scrum) via l'outil Jira, nous permettant de piloter le projet par sprints. Sur le plan technique, nous avons exploité une stack moderne et robuste : GitHub pour le versioning et le travail collaboratif, VS Code pour le développement, et des environnements de tests locaux comme XAMPP. Le choix de frameworks comme Laravel (PHP) et React (JS) a permis de structurer une solution professionnelle et maintenable.

Respect de la réglementation et des recommandations La sécurité et le respect de la vie privée ont été au cœur de nos préoccupations. Nous avons veillé à la conformité avec le RGPD et la LPD/nLPD suisse, notamment via la minimisation des données collectées et l'anonymisation des profils. De plus, nous avons anticipé les exigences de l'IA Act en classant notre système en "Haut Risque" et en intégrant des principes de transparence et de supervision humaine. Nous avons également suivi les bonnes pratiques de développement (Clean Code) et de sécurité pour protéger les échanges entre les serveurs PHP et Python.

Optimisation des ressources matérielles et énergétiques Conscients de l'impact environnemental du numérique, nous avons appliqué les recommandations du RGESN. Cela s'est traduit par des choix techniques concrets : utilisation de modèles d'IA légers (Small Language Models) moins énergivores, mise en place de systèmes de cache pour réduire la charge serveur, et une interface utilisateur sobre évitant les transferts de données inutiles (pagination, absence de vidéos, polices limitées).

Collaboration efficace La réussite du projet repose sur une coordination fluide au sein de l'équipe. Nous avons utilisé un Git Flow rigoureux (gestion de branches par fonctionnalités et revues de code) pour éviter les régressions. La communication a été maintenue quotidiennement via Microsoft Teams, permettant un partage de connaissances constant et une résolution rapide des points de blocage, notamment lors de l'intégration complexe du pôle IA.

Respect des spécifications Chaque étape du développement a été guidée par les spécifications techniques établies avec WeJob. Nous avons dû justifier nos choix de conception à plusieurs reprises, notamment lors du pivot de l'approche vectorielle vers un matching simplifié. Ce choix était motivé par une analyse pragmatique de la base de données héritée, nous permettant de livrer une solution fonctionnelle, précise et performante plutôt qu'un concept théorique inadapté à l'existant.

Justification des choix de conception L'intégrité technique du projet a reposé sur des arbitrages constants entre ambition technologique et viabilité opérationnelle :

- **Rationalisation du pipeline d'extraction** : Nous avons choisi d'omettre l'agent « Juge » lors de l'extraction des CV. Ce choix optimise la latence et les coûts d'inférence, tout en intégrant une approche « Human-in-the-Loop ». En plaçant l'utilisateur au centre de la validation des données extraites, l'agent automatisé devenait redondant, garantissant ainsi une fiabilité totale et une transparence accrue.
- **Approche incrémentale du Matching** : Nous avons privilégié un matching simplifié basé sur des indicateurs clairs (compétences, expérience, localisation). Ce choix pragmatique permet de valider la proposition de valeur auprès des utilisateurs finaux avec une grande explicabilité algorithmique, évitant les biais des modèles complexes avant d'avoir consolidé le MVP.
- **Refonte structurelle de la base de données** : Face aux limites de l'existant, nous avons procédé à une restructuration profonde des schémas. Cet investissement était indispensable pour assurer la scalabilité du produit, fluidifier les échanges entre les services (PHP/Python) et permettre l'ajout agile de nouvelles fonctionnalités sans accumuler de dette technique.

Adaptation à un environnement technologique et stratégique en évolution Ce projet nous a confrontés à une dette technique importante (base de données obsolète, infrastructure non dimensionnée pour l'IA). Nous avons su nous adapter en révisant notre planification et en procédant à une refonte structurelle nécessaire. Par ailleurs, l'évolution rapide des outils d'IA (comme l'émergence de Docling) nous a obligés à une veille technologique constante pour proposer la solution d'extraction de CV la plus efficace du marché, malgré la complexité des formats actuels.

Conclusion

WeJob réussit le pari d'allier performance technique et responsabilité éthique. En automatisant l'extraction de données complexes et en proposant un matching transparent, nous améliorons significativement l'expérience des candidats et des recruteurs, tout en favorisant la montée en compétences via les offres de formation.

Bibliographie

- ***[1] Documentation PHP***
- ***[2] Documentation Laravel***
- ***[3] Documentation JavaScript***
- ***[4] Documentation React***
- ***[5] Documentation Python***
- ***[6] Docling Library Documentation***
- ***[7] Référentiel Général d'Éco-conception de Services Numériques (RGESN)***
- ***[8] Réglementation européenne sur l'usage éthique de l'IA (IA Act)***
- ***[9] Réglementation Général de la Protection des Données (RGPD)***
- ***[10] Loi fédérale sur la Protection des Données (LPD)***
- ***[11] Nouvelle Loi fédérale sur la Protection des Données (nLPD)***